

Iterative solvers for the 3D periodic SEM Poisson problem in Jexpresso: SEM AMG, SC AMG, SC p-MG GMG, p-MG AMG and p-MG GMG

Jexpresso — `sm/elementLearning`

1 The discrete problem

The deck `problems/Elliptic/poisson_periodic_sem_3d` solves $-\nabla^2 u = f$ on the periodic box $\Omega = [0, 2\pi]^3$ with `sem_setup`. The mesh has `nelem` = n_e^3 hexahedra, each with `ngl` = $N + 1$ Legendre–Gauss–Lobatto (LGL) points ξ_i per direction, with weights ω_i . The basis is the tensor product of the 1D Lagrange polynomials $\psi_i(\xi)$, whose derivatives $\psi'_i(\xi_k)$ are Jexpresso’s `basis.dpsi[i,k]`. Local node (i, j, k) of element `iel` has global number `connijk[iel, i, j, k]`.

The quadrature is collocated (`Inexact`, $Q = N$), so the mass matrix is diagonal:

$$\mathbf{M}_I = \sum_{\text{iel}} \sum_{(i,j,k) \rightarrow I} \omega_i \omega_j \omega_k J_{ijk}^e, \quad (1)$$

where J_{ijk}^e is the Jacobian of element `iel` at node (i, j, k) (Jexpresso’s `metrics.Je`). The stiffness matrix is the direct stiffness summation (DSS) of the element matrices (`DSS_laplace_sparse_3D`):

$$\mathbf{K}_{IJ} = \sum_{\text{iel}} \sum_{l,m,n} \omega_l \omega_m \omega_n J_{lmn}^e \nabla \psi_I(\xi_l, \eta_m, \zeta_n) \cdot \nabla \psi_J(\xi_l, \eta_m, \zeta_n). \quad (2)$$

Here $\nabla = (\xi_x \partial_\xi + \eta_x \partial_\eta + \zeta_x \partial_\zeta, \dots)$, with the inverse metric terms $d\xi dx$, ..., $d\zeta dz$. Periodicity identifies the nodes of opposite faces into classes. The solve uses the class numbering, $\mathbf{K} = P^T L P$ (`periodic_sem_system`), and $\mathbf{K}$ is singular, with the constants in its null space. The right-hand side $\mathbf{b} = \mathbf{M}f$ is compatible ($\mathbf{1}^T \mathbf{b} = 0$), and the solution is fixed by a zero $\mathbf{M}$ -weighted mean.

Tensor structure on box elements. On an axis-aligned box element the metric terms are constant, $\xi_y = \xi_z = \eta_x = \dots = 0$. Equation (2) then factors into 1D matrices:

$$A_{\text{iel}} = c_x^{\text{iel}} \hat{\omega} \otimes \hat{\omega} \otimes \hat{K} + c_y^{\text{iel}} \hat{\omega} \otimes \hat{K} \otimes \hat{\omega} + c_z^{\text{iel}} \hat{K} \otimes \hat{\omega} \otimes \hat{\omega}, \quad (3)$$

(the index i runs fastest). Here

$$\hat{K}_{ab} = \sum_k \omega_k \psi'_a(\xi_k) \psi'_b(\xi_k), \quad \hat{\omega} = \text{diag}(\omega_k), \quad c_x^{\text{iel}} = J^e \xi_x^2, \quad c_y^{\text{iel}} = J^e \eta_y^2, \quad c_z^{\text{iel}} = J^e \zeta_z^2. \quad (4)$$

Because $\hat{\omega}$ is diagonal, each row of A_{iel} couples a node only to the nodes on the three grid lines through it. After assembly a row of $\mathbf{K}$ has between $3N + 1$ nonzeros (a node inside an element) and $6N + 1$ (an element vertex, whose three lines each cross two elements). Four of the five solvers below (SC AMG, SC p-MG GMG and the two p-MG solvers) exploit (3).

All five solvers run preconditioned conjugate gradients (CG) on the same matrix $\mathbf{K}$. They stop at a relative preconditioned residual $\sqrt{\mathbf{r}^T \mathbf{z}} / \sqrt{\mathbf{r}_0^T \mathbf{z}_0} \leq 10^{-12}$ and agree with the sparse direct solve to round-off (`verify_3d_jexpresso.jl`). They differ only in the preconditioner, and in whether CG works on all of $\mathbf{K}$ or only on its skeleton.

2 The solvers

2.1 SEM AMG (`sem_amg, :linsolve_amg => true`)

- **Operator:** the full system $\mathbf{K}\mathbf{u} = \mathbf{b}$, with one class pinned: CG on $\mathbf{K}_{2:,2:}$ (`periodic_sem_amg_solve`).
- **Preconditioner:** one V-cycle of smoothed-aggregation AMG (`AlgebraicMultigrid.jl`, `jx_amg_setup`), built from the entries of $\mathbf{K}$ alone.
- **Cost:** the matrix–vector product in CG is threaded (`JXSymCSC`); the AMG setup and V-cycle are serial.

AMG knows nothing about the SEM. Aggregation-based AMG is designed for M-matrices with connectivity like a low-order stencil. The high-order matrix $\mathbf{K}$ is different:

- the GLL stiffness $\hat{K}$ has positive off-diagonal entries;
- the strongest couplings run along the GLL lines;
- the nodes cluster near element boundaries, $\Delta\xi \sim N^{-2}$.

The aggregates therefore approximate the smooth modes poorly, and the iteration count grows with N : 29 at 4^3 elements, $N = 4$; 63 at 8^3 , $N = 6$; 59 at 16^3 , $N = 4$.

2.2 SC AMG (`sc_amg, :lstatic_condensation => true, :EL_skeleton_solver => "amg"`)

Split the unknowns into element interiors o ($(N-1)^3$ per element) and the skeleton b : the element-boundary nodes, ∂O in the element-learning notation. Eliminating the interiors gives the Schur complement on the skeleton:

$$S \mathbf{u}_b = \mathbf{b}_b - \sum_{\text{iel}} A_{bo} A_{oo}^{-1} \mathbf{b}_o, \quad S = A_{bb} - \sum_{\text{iel}} A_{bo} A_{oo}^{-1} A_{ob}, \quad \mathbf{u}_o = A_{oo}^{-1} (\mathbf{b}_o - A_{ob} \mathbf{u}_b). \quad (5)$$

Here A_{oo} is the interior block of element `iel` (`Jexpresso`’s `Avovo`), and A_{ob} the interior–boundary block (`Avovb`). The default implementation, `el_sc_schur_cg!`, never forms S :

- **Interior solves by fast diagonalization.** Take the interior part of (3), $A_{oo} = c_x \hat{\omega}_o \otimes \hat{\omega}_o \otimes \hat{K}_o + \dots$ (rows and columns $2:N$ of the 1D matrices). Solve the 1D generalized eigenproblem $\hat{K}_o s = \lambda \hat{\omega}_o s$, with $S_1^T \hat{\omega}_o S_1 = I$. Then

$$A_{oo}^{-1} = (S_1 \otimes S_1 \otimes S_1) \text{diag}\left(\frac{1}{c_x \lambda_i + c_y \lambda_j + c_z \lambda_k}\right) (S_1 \otimes S_1 \otimes S_1)^T, \quad (6)$$

which costs $O(N^4)$ per element. Only (c_x, c_y, c_z) is stored per element; these are read off $\mathbf{K}$ and checked entry by entry against (3).

- **Matrix-free Schur complement:** $S \mathbf{p} = (\mathbf{K} [\mathbf{p}; -A_{oo}^{-1} A_{ob} \mathbf{p}])_b$, one sparse matvec plus the interior solves (6).

- **Preconditioner:** the SEM AMG V-cycle of the *full* $\mathbf{K}$, P^{-1} , restricted to the skeleton: $\mathbf{z}_b = (P^{-1}[\mathbf{r}_b; 0])_b$. For an SPD matrix, $(\mathbf{K}^{-1})_{bb} = S^{-1}$ exactly. So if $c_1\mathbf{K} \leq P \leq c_2\mathbf{K}$, then $(P^{-1})_{bb}$ is spectrally equivalent to S^{-1} with the same constants, and $\kappa((P^{-1})_{bb}S) \leq \kappa(P^{-1}\mathbf{K})$.

SC AMG therefore inherits SEM AMG's convergence: 57 iterations against 59–63, a few fewer because CG runs on the smaller space. Each iteration costs one matvec, the interior solves and one full V-cycle. Its setup is the same AMG hierarchy as SEM AMG's.

The alternative, `:EL_sc_amg => "assembled"`, assembles S (`el_sc_solve!`) and gives it to AMG. That needs fewer iterations (34 at 8^3 , $N = 6$), but S is far denser than $\mathbf{K}$. A face node of the skeleton couples to all the active boundary nodes of both neighbouring elements, about 10 times the nonzeros of $\mathbf{K}$ at $N = 8$. Each iteration, and the setup, therefore cost much more.

2.3 p-MG + AMG and p-MG + GMG (`pmg_amg`, `pmg_gmg`, `:linsolve_pmg => "amg" | "gmg"`)

These run CG on the full singular $\mathbf{K}$, preconditioned by one p-multigrid V-cycle (`pmultigrid.jl`, `jx_pmg_setup`, `jx_pmg_cg`). The hierarchy follows the SEM order rather than the matrix entries.

p-levels. The orders are $p_0 = N$, $p_{\ell+1} = \lfloor p_\ell/2 \rfloor$, down to $p = 1$ (for example $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$), all on the same `nelem` elements:

- $\ell = 0$: $\mathbf{K}$ itself, built by `sem_setup`.
- $\ell > 0$: a rediscritization of the same operator with the LGL basis of order p , built by Jexpresso: `basis_structs_ξ_ω!(LGL(), p)` gives $\xi_k^{(p)}, \omega_k^{(p)}$; `build_Interpolation_basis!` gives $\psi_a^{(p)' }(\xi_k^{(p)})$; the coefficients c^{iel} come from `metrics`. $A^{(p)}$ is the DSS of (3) with $\widehat{K}^{(p)}, \widehat{\omega}^{(p)}$.
- **Check:** the order- N form of (3) must reproduce `diag(K)`, and each element's local axes must map to $\pm x, \pm y, \pm z$.

Transfers. Prolongation from order p_c to p_f is the tensor-product Lagrange interpolation

$$\Pi_{kl} = \psi_l^{(p_c)}(\xi_k^{(p_f)}), \quad \mathbf{u}_{\text{iel}}^{(p_f)} = (\Pi \otimes \Pi \otimes \Pi) \mathbf{u}_{\text{iel}}^{(p_c)}, \quad (7)$$

applied element by element in three 1D passes. Every fine node is written by one owning element. Restriction is the transpose. It accumulates element by element, with the elements coloured so that the threads never write the same coarse node.

Smoother. On every level, Chebyshev–Jacobi of degree 3 targets the upper part $[0.25 \lambda_{\max}, 1.1 \lambda_{\max}]$ of the spectrum of $D^{-1}A^{(p)}$. Here $D = \text{diag}(A^{(p)})$, and $\lambda_{\max}$ comes from 20 power iterations. Each Chebyshev step is

$$\mathbf{x} \leftarrow \mathbf{x} + \mathbf{d}, \quad \mathbf{r} \leftarrow \mathbf{r} - D^{-1}A\mathbf{d}, \quad \mathbf{d} \leftarrow \rho_{k+1}\rho_k \mathbf{d} + \frac{2\rho_{k+1}}{\delta} \mathbf{r}.$$

It needs only threaded matvecs, and it is a fixed polynomial in $D^{-1}A$, so pre- and post-smoothing keep the V-cycle symmetric, as CG requires.

Why this works for high order. A Jacobi-type smoother removes the components of the error that oscillate on the scale of the GLL spacing. What remains is well represented by a polynomial of half the order: the same operator, rediscritized with $p/2$, is spectrally close on that subspace. The hierarchy ends at $p = 1$: the trilinear SEM with two-point GLL (trapezoidal) quadrature. There, $\widehat{K}^{(1)} = \frac{1}{2} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$ and $\widehat{\omega}^{(1)} = I$, so (3) is the 7-point Laplacian on the element vertices: a symmetric M-matrix on an n_e^3 grid, exactly the problem AMG and geometric multigrid are designed for.

The $p = 1$ level: the only difference between the two variants.

- **p-MG + AMG:** one smoothed-aggregation AMG V-cycle on $A^{(1)}$, with one node pinned.
- **p-MG + GMG:** geometric h-multigrid on the periodic n_e^3 vertex grid:
 - trilinear interpolation P_h (weights 1 and $\frac{1}{2}$ per direction);
 - Galerkin coarse operators $A_{2h} = P_h^T A_h P_h$;
 - coarsening by 2 while every dimension is even and the grid has more than 512 nodes;
 - a sparse Cholesky solve of the pinned coarsest operator.

Both variants share the p-levels, so the iteration counts are nearly equal (12 against 10 at 16^3 , $N = 4$). GMG needs no aggregation setup and its transfers are known in advance, so its setup is cheaper. AMG needs no structured grid.

The singular system. CG runs on $\mathbf{K}$ itself. The V-cycle subtracts the mean from its input and output. Interpolation reproduces constants, so coarse right-hand sides stay compatible. The coarsest solves pin one node, which solves a compatible singular system exactly.

2.4 Side by side

	SEM AMG	SC AMG	SC p-MG GMG	p-MG + AMG	p-MG + GMG
CG operator	$\mathbf{K}$ (pinned)	S on the skeleton, matrix-free	as SC AMG	$\mathbf{K}$ (singular)	$\mathbf{K}$ (singular)
preconditioner	AMG V-cycle of $\mathbf{K}$	AMG V-cycle of $\mathbf{K}$, restricted to b	p-MG + GMG V-cycle of $\mathbf{K}$, restricted to b	p-V-cycle $N \rightarrow \dots \rightarrow 1$, AMG at $p = 1$	p-V-cycle $N \rightarrow \dots \rightarrow 1$, GMG at $p = 1$
uses the SEM structure	no	yes: A_{oo}^{-1} by fast diagonalization (6)	yes: (6), (3), (7)	yes: levels (3), transfers (7)	yes, and the vertex grid
smoother	Gauss–Seidel (AMG)	Gauss–Seidel (AMG)	Chebyshev–Jacobi	Chebyshev–Jacobi; AMG at $p = 1$	Chebyshev–Jacobi on all levels
threaded	matvec	matvec, interior solves	everything but the coarsest solve	all p-levels	all levels
needs	any SPD $\mathbf{K}$	box elements, else assembled S	box elements, periodic box	box elements, periodic box	same; h-levels while n_e stays even

Table 1: The five iterative solvers. “Box elements” means axis-aligned hexahedra with constant metric terms, the structure assumed by (3).

	8 ³ elements, $N = 6$ ($1.1 \cdot 10^5$ unknowns)			16 ³ elements, $N = 4$ ($2.6 \cdot 10^5$ unknowns)		
	iters	setup [s]	solve [s]	iters	setup [s]	solve [s]
SEM AMG	63	3.1	3.5	59	7.6	5.3
SC AMG	57	2.6	3.2	57	8.8	5.9
SC p-MG GMG	19	0.4	0.35	15	2.1	0.59
p-MG + AMG	13	1.3	0.8	12	3.2	1.1
p-MG + GMG	13	1.0	0.36	10	1.3–2.6	0.64–0.8

Table 2: Local measurements, 4 threads on a shared 4-core container; the timings are noisy, the iteration counts are not. “setup” includes the periodic reduction (0.3–2 s). Where two runs of the same configuration differ, the range is given. The Wulver sweep (`run_wulver3d.sbatch`) gives the definitive numbers.

3 Static condensation and geometric multigrid

3.1 Why there is no geometric multigrid of S

1. **Geometric multigrid needs a grid hierarchy for the operator it solves.** GMG rests on three geometric ingredients:

- a sequence of meshes on which the *same* discrete operator can be rebuilt;

- transfers that interpolate between them;
- a local smoother that removes the high-frequency error on each.

The p-MG levels and the $p = 1$ vertex grid have all three, because every level discretizes $-\nabla^2$ with the SEM. The Schur complement S in (5) has none of them:

- It lives only on the skeleton, the union of element faces, which is not a volume mesh.
- It is the discrete Steklov–Poincaré (Dirichlet-to-Neumann) operator of the elements: dense within each element’s boundary and nonlocal, with a condition number growing like h^{-1} rather than h^{-2} .
- A coarser skeleton is not the skeleton of a coarser SEM. Halving n_e turns half of the fine skeleton into interiors of the coarse elements, so the coarse Schur complement eliminates nodes that are unknowns on the fine level.
- The transfers between two skeleton levels must therefore include the discrete harmonic extension $-A_{oo}^{-1}A_{ob}$ into the eliminated interiors. That is substructuring (balancing domain decomposition, BDDC, Schur-complement multigrid), not geometric multigrid.
- A pointwise smoother on S needs $\text{diag}(S)$. That is only available element by element through $A_{bo}A_{oo}^{-1}A_{ob}$, the work the matrix-free path avoids.

A geometric multigrid applied to S itself is therefore a different method, not a variant of the ones implemented.

2. What can be done instead: restrict a geometric multigrid of $\mathbf{K}$. SC AMG never applies AMG to S . It restricts the full-system V-cycle, $\mathbf{z}_b = (P^{-1}[\mathbf{r}_b; 0])_b$. That is a valid preconditioner for S *whatever* SPD multigrid P is, because $(\mathbf{K}^{-1})_{bb} = S^{-1}$, and $\kappa((P^{-1})_{bb}S) \leq \kappa(P^{-1}\mathbf{K})$. Taking for P the p-MG + GMG V-cycle gives a static condensation with a geometric multigrid without ever building a hierarchy for S : the solver below.

3.2 SC p-MG GMG (`sc_pmg`, `:EL_sc_precond => "pmg_gmg"`)

- **Operator:** the same matrix-free Schur complement as SC AMG (`el_sc_schur_cg!`), with the interior solves (6) and the recovery $\mathbf{u}_o = A_{oo}^{-1}(\mathbf{b}_o - A_{ob}\mathbf{u}_b)$.
- **Preconditioner:** the p-MG + GMG V-cycle of the *full* singular $\mathbf{K}$ (`jx_pmg_setup`, coarse level `:gmg`), in $\mathbf{K}$ ’s own numbering, restricted to the skeleton unknowns:

$$\mathbf{z}_b = (\mathcal{V}_{\text{pMG}}[\mathbf{r}_b; 0_o; 0_{\text{pin}}])_b.$$

The pinned skeleton node and the interiors get a zero residual. The V-cycle’s mean projection is harmless: a zero-extended skeleton vector is never constant, so the restricted operator stays SPD.

- **Setup:** the p-multigrid hierarchy, built in `periodic_sem_sc_solve` because it needs the SEM mesh and metrics, and timed as the skeleton setup (`:sc_factor`). No AMG and no assembled S .
- **Code:** `_sc_pcg` takes any full-system preconditioner through `ldiv!`; `:EL_sc_precond => "pmg_amg"` uses the p-MG + AMG V-cycle instead.

Measured behaviour. SC p-MG GMG needs *more* iterations than p-MG + GMG on the full system: 19 against 13 at 8^3 , $N = 6$, and 15 against 10 at 16^3 , $N = 4$. The bound $\kappa((P^{-1})_{bb}S) \leq \kappa(P^{-1}\mathbf{K})$ allows that: it limits the condition number, not the clustering of the eigenvalues. For SC AMG the two counts were nearly equal (57 against 59–63). An iteration costs one V-cycle on the full grid, one sparse matvec and the interior solves (6), against one V-cycle and one matvec for p-MG + GMG. Even so, the measured solve times are on par (0.35 against 0.36 s at 8^3 , 0.59 against 0.64–0.8 s at 16^3), within the noise of these runs. So this is the most competitive static condensation in the comparison, at the level of the best full-system solver. It does not beat it, as expected: the preconditioner is the same, and condensation only removes interior unknowns that the Chebyshev smoother already handles cheaply. Condensation pays off more clearly in the SC *direct* solver, where it removes the interior fill from the factorization, and in element learning, where the per-element operators $T^{\text{iel}} = A_{oo}^{-1}A_{ob}$ are what the network predicts.

4 Reuse of the local operators on a regular mesh

On a uniform Cartesian mesh every element is the same box, so $c_x^{\text{iel}}, c_y^{\text{iel}}, c_z^{\text{iel}}$ in (3) are the same for every element, up to a permutation of the axes where the local orientation differs. Then A_{oo} , A_{ob} , A_{oo}^{-1} , $T^{\text{iel}} = A_{oo}^{-1}A_{ob}$ and $A_{bo}A_{oo}^{-1}A_{ob}$ are the same matrices, in local numbering, on every element. A single element's factorization could serve the whole domain. What each path does:

- **Schur-CG** (`el_sc_schur_cg!`: SC AMG, SC p-MG GMG) forms and inverts no local matrix. The only factorization is the 1D generalized eigenproblem $\hat{K}_{os} = \lambda \hat{\omega}_os$, solved once for the whole domain. Each element keeps three numbers c^{iel} , and A_{oo}^{-1} is *applied* by (6): $O(N^4)$ per element, necessarily per element because every element has its own data. The one redundant step is the setup check: each element's A_{oo} is extracted from $\mathbf{K}$ and compared entry by entry with (3).
- **Assembled condensation** (`el_sc_solve!`: SC direct, and SC AMG with `:EL_sc_amg => "assembled"`) does recompute on every element: it extracts A_{oo} and A_{ob} from $\mathbf{K}$ and repeats the Cholesky factorization (`potrf`), $W = L^{-1}A_{ob}$ (`trsm`) and W^TW (`syrk`), which are identical on a uniform mesh. This work is threaded and small next to the skeleton factorization: 0.004 s against 0.18 s at 4^3 , $N = 5$, and 0.09 s against 1.9 s (AMG setup of S) at 8^3 , $N = 6$. Computing it once per distinct (c_x, c_y, c_z) would save at most a few percent of these solvers' time.
- **The legacy element-learning loop** inverts A_{oo} with a dense `inv` on every element. This is the expensive version. It is used for sampling, where the coefficient $a(\mathbf{x})$ usually varies from element to element, so the local operators genuinely differ.